```
import pandas as pd
import numpy as np

data = {
    "Order_ID": [
        "ORD101", "ORD102", "ORD103", "ORD104", "ORD105",
        "ORD106", "ORD107", "ORD108", "ORD109", "ORD110",
        "ORD111", "ORD112", "ORD113", "ORD114", "ORD115"
    ],
    "City": [
        "Chennai", "chennai", " CHENNAI ", "Bengaluru", "BENGALURU",
        "Mumbai", "mumbai ", None, "Hyderabad", "HYDERABAD",
        "Chennai", " Delhi ", "delhi", "Mumbai", "Chennai"
    ],
    "Product": [
        "Laptop", "Mouse", "Keyboard", "Laptop", "Monitor",
        "Phone", "Mouse", "Tablet", "Laptop", "Phone",
        "Monitor", "Keyboard", "Tablet", "Laptop", "Mouse"
    ],
    "Quantity": [
        "2", "5", "3", "2", "1", "4", "6", "2",
        "3", "2", None, "4", "3", "2", "5"
    ],
    "Unit_Price": [
        "65000", "500", "1500", "65000", "12000",
        "25000", "500", "18000", "65000", "25000",
        "12000", "1500", "18000", "65000", "500"
    ],
    "Sales": [
        130000, 2500, 4500, 130000, 12000,
        100000, 3000, 36000, 195000, 50000,
        12000, 6000, 54000, 130000, 2500
    ],
    "Order_Date": [
        "2026-10-01", "01/10/2026", "2026-10-03", "04-10-2026",
        "2026/10/05", "06/10/2026", "2026-10-07", "08-10-2026",
        "2026/10/09", "10/10/2026", "2026-10-11", "12/10/2026",
        "2026-10-13", "14-10-2026", "2026/10/15"
    ],
    "Customer_Review": [
        "Good", " good ", "EXCELLENT", "Average", None,
        "Good", "Poor", "Excellent", " good", "Average",
        "Excellent", "POOR", "Good", "Average", "Excellent"
    ]
}

df = pd.DataFrame(data)

# Introduce practice data-quality issues
df.loc[4, "Sales"] = np.nan
df.loc[7, "Sales"] = 9999999
df.loc[11, "Sales"] = -5000
df.loc[14, "Quantity"] = "invalid"

print("Original DataFrame:")
display(df)

print("Rows and columns:", df.shape)
```

```
Original DataFrame:
```

	Order_ID	City	Product	Quantity	Unit_Price	Sales	Order_Date	Customer_Review
0	ORD101	Chennai	Laptop	2	65000	130000.0	2026-10-01	Good
1	ORD102	chennai	Mouse	5	500	2500.0	01/10/2026	good
2	ORD103	CHENNAI	Keyboard	3	1500	4500.0	2026-10-03	EXCELLENT
3	ORD104	Bengaluru	Laptop	2	65000	130000.0	04-10-2026	Average
4	ORD105	BENGALURU	Monitor	1	12000	NaN	2026/10/05	None
5	ORD106	Mumbai	Phone	4	25000	100000.0	06/10/2026	Good
6	ORD107	mumbai	Mouse	6	500	3000.0	2026-10-07	Poor
7	ORD108	None	Tablet	2	18000	9999999.0	08-10-2026	Excellent
8	ORD109	Hyderabad	Laptop	3	65000	195000.0	2026/10/09	good
9	ORD110	HYDERABAD	Phone	2	25000	50000.0	10/10/2026	Average
10	ORD111	Chennai	Monitor	None	12000	12000.0	2026-10-11	Excellent
11	ORD112	Delhi	Keyboard	4	1500	-5000.0	12/10/2026	POOR
12	ORD113	delhi	Tablet	3	18000	54000.0	2026-10-13	Good
13	ORD114	Mumbai	Laptop	2	65000	130000.0	14-10-2026	Average
14	ORD115	Chennai	Mouse	invalid	500	2500.0	2026/10/15	Excellent

```
Rows and columns: (15, 8)
```

Next steps: Generate code with df | New interactive sheet

```
df_dropped = df.dropna()

print("Original row count:", len(df))
print("Rows after dropna():", len(df_dropped))

display(df_dropped)
```

```
Original row count: 15
Rows after dropna(): 12
```

	Order_ID	City	Product	Quantity	Unit_Price	Sales	Order_Date	Customer_Review
0	ORD101	Chennai	Laptop	2	65000	130000.0	2026-10-01	Good
1	ORD102	chennai	Mouse	5	500	2500.0	01/10/2026	good
2	ORD103	CHENNAI	Keyboard	3	1500	4500.0	2026-10-03	EXCELLENT
3	ORD104	Bengaluru	Laptop	2	65000	130000.0	04-10-2026	Average
5	ORD106	Mumbai	Phone	4	25000	100000.0	06/10/2026	Good
6	ORD107	mumbai	Mouse	6	500	3000.0	2026-10-07	Poor
8	ORD109	Hyderabad	Laptop	3	65000	195000.0	2026/10/09	good
9	ORD110	HYDERABAD	Phone	2	25000	50000.0	10/10/2026	Average
11	ORD112	Delhi	Keyboard	4	1500	-5000.0	12/10/2026	POOR
12	ORD113	delhi	Tablet	3	18000	54000.0	2026-10-13	Good
13	ORD114	Mumbai	Laptop	2	65000	130000.0	14-10-2026	Average
14	ORD115	Chennai	Mouse	invalid	500	2500.0	2026/10/15	Excellent

Next steps: Generate code with df_dropped | New interactive sheet

```
df_filled = df.copy()

df_filled["City"] = df_filled["City"].fillna("Unknown")
df_filled["Customer_Review"] = df_filled["Customer_Review"].fillna("Not Provided")
df_filled["Sales"] = df_filled["Sales"].fillna(df_filled["Unit_Price"].astype(float) * pd.to_numeric(df_filled["Quantity"],

print("Missing values after filling:")
print(df_filled.isnull().sum())

display(df_filled)
```

```
Missing values after filling:
Order_ID          0
City              0
Product           0
Quantity          1
Unit_Price        0
Sales             0
Order_Date        0
Customer_Review   0
dtype: int64
```

	Order_ID	City	Product	Quantity	Unit_Price	Sales	Order_Date	Customer_Review
0	ORD101	Chennai	Laptop	2	65000	130000.0	2026-10-01	Good
1	ORD102	chennai	Mouse	5	500	2500.0	01/10/2026	good
2	ORD103	CHENNAI	Keyboard	3	1500	4500.0	2026-10-03	EXCELLENT
3	ORD104	Bengaluru	Laptop	2	65000	130000.0	04-10-2026	Average
4	ORD105	BENGALURU	Monitor	1	12000	12000.0	2026/10/05	Not Provided
5	ORD106	Mumbai	Phone	4	25000	100000.0	06/10/2026	Good
6	ORD107	mumbai	Mouse	6	500	3000.0	2026-10-07	Poor
7	ORD108	Unknown	Tablet	2	18000	9999999.0	08-10-2026	Excellent
8	ORD109	Hyderabad	Laptop	3	65000	195000.0	2026/10/09	good
9	ORD110	HYDERABAD	Phone	2	25000	50000.0	10/10/2026	Average
10	ORD111	Chennai	Monitor	None	12000	12000.0	2026-10-11	Excellent
11	ORD112	Delhi	Keyboard	4	1500	-5000.0	12/10/2026	POOR
12	ORD113	delhi	Tablet	3	18000	54000.0	2026-10-13	Good
13	ORD114	Mumbai	Laptop	2	65000	130000.0	14-10-2026	Average
14	ORD115	Chennai	Mouse	invalid	500	2500.0	2026/10/15	Excellent

Next steps: Generate code with `df_filled` New interactive sheet

```
# Convert sales temporarily to numeric for calculations
sales_before = pd.to_numeric(df["Sales"], errors="coerce")

print("Total sales:", sales_before.sum())
print("Average sales:", sales_before.mean())
print("Median sales:", sales_before.median())

quantity_before = pd.to_numeric(df["Quantity"], errors="coerce")

print("Total quantity:", quantity_before.sum())
print("Average quantity:", quantity_before.mean())
print("Median quantity:", quantity_before.median())
```

```
Total sales: 10808499.0
Average sales: 772035.6428571428
Median sales: 52000.0
Total quantity: 39.0
Average quantity: 3.0
Median quantity: 3.0
```

```
df_clean = df.copy()

# Remove spaces and standardize city names
df_clean["City"] = (
    df_clean["City"]
    .str.strip()
    .str.title()
    .fillna("Unknown")
)

# Standardize product names
df_clean["Product"] = df_clean["Product"].str.strip().str.title()

# Standardize review text
df_clean["Customer_Review"] = (
    df_clean["Customer_Review"]
    .str.strip()
    .str.title()
    .fillna("Not Provided")
)

# Convert dates with mixed formats
```

```
df_clean["Order_Date"] = pd.to_datetime(
    df_clean["Order_Date"],
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

print(df_clean[["City", "Product", "Customer_Review", "Order_Date"]])
```

```
         City   Product Customer_Review Order_Date
0     Chennai    Laptop            Good 2026-10-01
1     Chennai     Mouse            Good 2026-10-01
2     Chennai  Keyboard       Excellent 2026-10-03
3   Bengaluru    Laptop         Average 2026-10-04
4   Bengaluru   Monitor    Not Provided 2026-10-05
5      Mumbai     Phone            Good 2026-10-06
6      Mumbai     Mouse            Poor 2026-10-07
7     Unknown    Tablet       Excellent 2026-10-08
8   Hyderabad    Laptop            Good 2026-10-09
9   Hyderabad     Phone         Average 2026-10-10
10    Chennai   Monitor       Excellent 2026-10-11
11      Delhi  Keyboard            Poor 2026-10-12
12      Delhi    Tablet            Good 2026-10-13
13     Mumbai    Laptop         Average 2026-10-14
14    Chennai     Mouse       Excellent 2026-10-15
```

```
# Convert numeric columns stored as text
df_clean["Quantity"] = pd.to_numeric(
    df_clean["Quantity"],
    errors="coerce"
)

df_clean["Unit_Price"] = pd.to_numeric(
    df_clean["Unit_Price"],
    errors="coerce"
)

df_clean["Sales"] = pd.to_numeric(
    df_clean["Sales"],
    errors="coerce"
)

# Convert ID to string type
df_clean["Order_ID"] = df_clean["Order_ID"].astype("string")

print("Data types:")
print(df_clean.dtypes)

print("\nDataFrame information:")
df_clean.info()
```

```
Data types:
Order_ID           string[python]
City                       object
Product                    object
Quantity                  float64
Unit_Price                  int64
Sales                     float64
Order_Date         datetime64[ns]
Customer_Review            object
dtype: object

DataFrame information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Order_ID         15 non-null     string
 1   City             15 non-null     object
 2   Product          15 non-null     object
 3   Quantity         13 non-null     float64
 4   Unit_Price       15 non-null     int64
 5   Sales            14 non-null     float64
 6   Order_Date       15 non-null     datetime64[ns]
 7   Customer_Review  15 non-null     object
dtypes: datetime64[ns](1), float64(2), int64(1), object(3), string(1)
memory usage: 1.1+ KB
```

```
print(df_clean[["Quantity", "Unit_Price", "Sales"]].describe())
```

```
        Quantity    Unit_Price         Sales
count  13.000000     15.000000  1.400000e+01
mean    3.000000  24966.666667  7.720356e+05
std     1.414214  26378.066285  2.656773e+06
```

```
min     1.000000    500.000000 -5.000000e+03
25%     2.000000   1500.000000  3.375000e+03
50%     3.000000  18000.000000  5.200000e+04
75%     4.000000  45000.000000  1.300000e+05
max     6.000000  65000.000000  9.999999e+06
```

```
Q1 = df_clean["Sales"].quantile(0.25)
Q3 = df_clean["Sales"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)

outliers = df_clean[
    (df_clean["Sales"] < lower_bound) |
    (df_clean["Sales"] > upper_bound)
]

print("\nPotential sales outliers:")
display(outliers)
```

```
Q1: 3375.0
Q3: 130000.0
IQR: 126625.0
Lower bound: -186562.5
Upper bound: 319937.5

Potential sales outliers:
```

	Order_ID	City	Product	Quantity	Unit_Price	Sales	Order_Date	Customer_Review
7	ORD108	Unknown	Tablet	2.0	18000	9999999.0	2026-10-08	Excellent

```
# Save the pre-cleaning version for comparison
df_before_outlier_handling = df_clean.copy()

# Handle invalid quantity
# Use the median only as a practice example; verify real records first.
quantity_median = df_clean["Quantity"].median()
df_clean["Quantity"] = df_clean["Quantity"].fillna(quantity_median)

# Calculate expected sales from quantity and unit price
expected_sales = df_clean["Quantity"] * df_clean["Unit_Price"]

# Recalculate sales when missing, negative, or above the IQR upper bound
invalid_sales = (
    df_clean["Sales"].isna() |
    (df_clean["Sales"] < 0) |
    (df_clean["Sales"] > upper_bound)
)

df_clean.loc[invalid_sales, "Sales"] = expected_sales[invalid_sales]

print("Sales after handling identified issues:")
display(df_clean[["Order_ID", "Quantity", "Unit_Price", "Sales"]])
```

```
Sales after handling identified issues:
```

	Order_ID	Quantity	Unit_Price	Sales
0	ORD101	2.0	65000	130000.0
1	ORD102	5.0	500	2500.0
2	ORD103	3.0	1500	4500.0
3	ORD104	2.0	65000	130000.0
4	ORD105	1.0	12000	12000.0
5	ORD106	4.0	25000	100000.0
6	ORD107	6.0	500	3000.0
7	ORD108	2.0	18000	36000.0
8	ORD109	3.0	65000	195000.0
9	ORD110	2.0	25000	50000.0
10	ORD111	3.0	12000	12000.0
11	ORD112	4.0	1500	6000.0
12	ORD113	3.0	18000	54000.0
13	ORD114	2.0	65000	130000.0
14	ORD115	3.0	500	2500.0

```
print("Original rows:", len(df))
print("Cleaned rows:", len(df_clean))

comparison = pd.DataFrame({
    "Missing Before": df.isnull().sum(),
    "Missing After": df_clean.isnull().sum()
})

print(comparison)
```

```
Original rows: 15
Cleaned rows: 15
                Missing Before  Missing After
Order_ID                     0              0
City                         1              0
Product                      0              0
Quantity                     1              0
Unit_Price                   0              0
Sales                        1              0
Order_Date                   0              0
Customer_Review              1              0
```

```
stats_before = pd.DataFrame({
    "Before Cleaning": [
        sales_before.sum(),
        sales_before.mean(),
        sales_before.median()
    ]
}, index=["Total Sales", "Average Sales", "Median Sales"])

stats_after = pd.DataFrame({
    "After Cleaning": [
        df_clean["Sales"].sum(),
        df_clean["Sales"].mean(),
        df_clean["Sales"].median()
    ]
}, index=["Total Sales", "Average Sales", "Median Sales"])

stats_comparison = stats_before.join(stats_after)

print(stats_comparison)
```

```
              Before Cleaning  After Cleaning
Total Sales       1.080850e+07   867500.000000
Average Sales     7.720356e+05    57833.333333
Median Sales      5.200000e+04    36000.000000
```

```
print("Final cleaned dataset:")
display(df_clean)

print("\nFinal data types:")
print(df_clean.dtypes)
```

```
print("\nFinal missing-value counts:")
print(df_clean.isnull().sum())
```

```
Final cleaned dataset:
```

	Order_ID	City	Product	Quantity	Unit_Price	Sales	Order_Date	Customer_Review
0	ORD101	Chennai	Laptop	2.0	65000	130000.0	2026-10-01	Good
1	ORD102	Chennai	Mouse	5.0	500	2500.0	2026-10-01	Good
2	ORD103	Chennai	Keyboard	3.0	1500	4500.0	2026-10-03	Excellent
3	ORD104	Bengaluru	Laptop	2.0	65000	130000.0	2026-10-04	Average
4	ORD105	Bengaluru	Monitor	1.0	12000	12000.0	2026-10-05	Not Provided
5	ORD106	Mumbai	Phone	4.0	25000	100000.0	2026-10-06	Good
6	ORD107	Mumbai	Mouse	6.0	500	3000.0	2026-10-07	Poor
7	ORD108	Unknown	Tablet	2.0	18000	36000.0	2026-10-08	Excellent
8	ORD109	Hyderabad	Laptop	3.0	65000	195000.0	2026-10-09	Good
9	ORD110	Hyderabad	Phone	2.0	25000	50000.0	2026-10-10	Average
10	ORD111	Chennai	Monitor	3.0	12000	12000.0	2026-10-11	Excellent
11	ORD112	Delhi	Keyboard	4.0	1500	6000.0	2026-10-12	Poor
12	ORD113	Delhi	Tablet	3.0	18000	54000.0	2026-10-13	Good
13	ORD114	Mumbai	Laptop	2.0	65000	130000.0	2026-10-14	Average
14	ORD115	Chennai	Mouse	3.0	500	2500.0	2026-10-15	Excellent

```
Final data types:
```